

ARCHITECTURE

Architecture

Component Diagram

```
graph TB
    subgraph LLMOrchestrator
        Pool[AgentPool]
        HM[HealthMonitor]
        CB[CircuitBreaker]

        subgraph Adapters
            OC[OpenCodeAgent]
            CC[ClaudeCodeAgent]
            GEM[GeminiAgent]
            JUN[JunieAgent]
            QW[QwenCodeAgent]
        end
    end

    BA[BaseAdapter]
    PT[PipeTransport]
    FT[FileTransport]
    RP[ResponseParser]
    CFG[Config]

    Pool --> HM
    HM --> CB
    OC --> BA
    CC --> BA
    GEM --> BA
    JUN --> BA
    QW --> BA
    BA --> PT
    BA --> RP
    BA --> FT
    CFG --> Pool
```

Sequence Diagram: Agent Send

```
sequenceDiagram
    participant C as Caller
    participant P as AgentPool
    participant A as Agent (Adapter)
    participant CB as CircuitBreaker
    participant T as PipeTransport
    participant CLI as CLI Process

    C->>P: Acquire(ctx, requirements)
    P->>P: findAvailable(requirements)
    P-->>C: agent
    C->>A: Send(ctx, prompt)
    A->>CB: AllowRequest()
    CB-->>A: true
    A->>T: SendPrompt(ctx, id, prompt, "")
    T->>CLI: JSON-line via stdin
    CLI-->>T: JSON-line via stdout
    T-->>A: PipeMessage
    A->>CB: RecordSuccess()
    A-->>C: Response
    C->>P: Release(agent)
```

Class Diagram

```
classDiagram
    class Agent {
        <<interface>>
        +ID() string
        +Name() string
        +Start(ctx) error
        +Stop(ctx) error
        +IsRunning() bool
        +Health(ctx) HealthStatus
        +Send(ctx, prompt) Response, error
        +SendStream(ctx, prompt) chan StreamChunk, error
        +SendWithAttachments(ctx, prompt, attachments) Response, error
    }

    class BaseAdapter {
        -id string
        -name string
        -config AdapterConfig
        -transport PipeTransport
    }
```

```

        -breaker CircuitBreaker
        +Start(ctx) error
        +Stop(ctx) error
        +Send(ctx, prompt) Response, error
    }

    class AgentPool {
        <<interface>>
        +Register(agent) error
        +Acquire(ctx, requirements) Agent, error
        +Release(agent)
        +Available() []Agent
        +HealthCheck(ctx) []HealthStatus
        +Shutdown(ctx) error
    }

    Agent <|.. BaseAdapter
    BaseAdapter <|-- OpenCodeAgent
    BaseAdapter <|-- ClaudeCodeAgent
    BaseAdapter <|-- GeminiAgent
    BaseAdapter <|-- JunieAgent
    BaseAdapter <|-- QwenCodeAgent

```

State Diagram: Circuit Breaker

```

stateDiagram-v2
    [*] --> Closed
    Closed --> Open : N consecutive failures
    Open --> HalfOpen : Recovery timeout elapsed
    HalfOpen --> Closed : Success
    HalfOpen --> Open : Failure
    Closed --> Closed : Success (reset counter)

```

Flowchart: Agent Acquisition

```

flowchart TD
    A[Acquire Request] --> B{Pool Shutdown?}
    B -->|Yes| C[Return ErrPoolShutdown]
    B -->|No| D{Context Cancelled?}
    D -->|Yes| E[Return ctx.Err]
    D -->|No| F{Preferred Agent Available?}
    F -->|Yes| G{Meets Requirements?}
    G -->|Yes| H[Return Preferred Agent]
    G -->|No| I{Any Agent Meets Requirements?}
    F -->|No| I
    I -->|Yes| J[Return Matching Agent]
    I -->|No| K[Wait on Condition Variable]
    K --> B

```

Key Design Decisions

1. **BaseAdapter Pattern:** Shared process management (spawn, pipe, shutdown) in BaseAdapter. Each CLI adapter only implements protocol-specific response parsing.
2. **Thread-safe Pool:** Uses `sync.Mutex` + `sync.Cond` for blocking acquire with context cancellation support.
3. **Circuit Breaker:** Per-agent, 3 consecutive failures opens the circuit for 60 seconds. Half-open state allows probe requests.
4. **Hybrid Communication:** Real-time pipe for interactive prompts, file-based for large artifacts. Prevents blocking on large payloads.
5. **No Module Dependencies:** LLMOrchestrator defines its own types (`ModelInfo`, etc.) and does not import `LLMsVerifier`, `VisionEngine`, or `DocProcessor`. HelixQA bridges them.